

SHARP PC-1251 エミュレータ

使い方の手引き



おりぐち

2026 年 9 月 24 日

このエミュレータは PC-1251 の実物の ROM をそのまま動かします。ROM は付属していません (2.4 節)。

目次

1	はじめに	2
2	入れ方と起動	2
2.1	端末を開く	2
2.2	uv を入れる	3
2.3	エミュレータのフォルダを用意する	3
2.4	ROM を用意する	4
2.5	起動する	5
2.6	終わる	5
2.7	うまく動かないとき	5
3	画面と操作	6
3.1	各部の名前	6
3.2	マウスで操作する	7
3.3	パソコンのキーボードで操作する	7
3.4	エミュレータの操作	7
3.5	モードスイッチ	9
3.6	SHIFT と DEF	9
4	使ってみる	9
4.1	電卓として使う	9
4.2	BASIC のプログラムを打ち込んで動かす	9
4.3	打ち込みを楽にする、行を直す・消す	10
4.4	PRINT で止めずに見せる (WAIT)	11
4.5	DEF キーで実行する	12
4.6	RSV モードで文字列を登録する	12
4.7	エラーが出たら	13
4.8	電源とメモリ	13
5	プログラムのファイルを使う	14
5.1	置き場所に置いて、一覧から読み込む	14
5.2	ファイルの形式	15
5.3	端末から読み込む	16
5.4	マシン語を動かす	16
5.5	CALL で表示をつけたまま計算する	16
5.6	マシン語で液晶に描く	17
5.7	PC-インタープリタともぐらたたき	18
5.8	そのほかの読み込み方	19
5.9	プログラムをファイルに書き出す	19
付録 A	起動のオプション	20
付録 B	SHIFT で入る記号	20
付録 C	しくみのあらまし	21
付録 D	実機と違うところ	21
付録 E	ファイルの一覧	22

1 はじめに

このエミュレータは、1980 年代のはじめにシャープが出したポケットコンピュータ PC-1251 を、パソコンの上で動かすものです。CPU の SC61860 を命令単位で動かし、実機の内部 ROM と BASIC ROM をそのまま実行します。液晶と本体は、寸法を測り、写真から色を採取して描画しています。キーとモードスイッチはマウスでも操作できます。

BASIC のプログラムを打ち込んで動かせるほか、電卓としても使えます。BEEP の音も鳴ります。BASIC のテキストや 16 進ダンプのファイルを決まったフォルダに置いておけば、一覧から選んで読み込めますし、マシン語を RAM に書いて CALL で呼ぶこともできます。実機の「MEMORY SAFE GUARD」と同じく、終了してもプログラムは消えません。

動作環境

- macOS、Windows(10 以降)、Linux。
- Python 3.10 以降。Python を自分で入れる必要はありません。次の節で入れる uv が、必要な版の Python とライブラリをまとめて用意します。
- 初回の起動でライブラリを入れるときだけ、インターネットにつながっている必要があります。

この手引きの読み方

パソコンのキーは `Esc`、PC-1251 のキーは `SHIFT` のように書き分けます。 `command` / `Ctrl` + `O` は、macOS では `command` を、Windows と Linux では `Ctrl` を押しながら `O` を押すという意味です。macOS と Windows で違うところは「macOS」「Windows」と見出しをつけて両方を書きました。端末 (macOS の「ターミナル」、Windows の「PowerShell」) で打つコマンドは、次のような枠に入れています。行の頭の `$` や `>` は打たなくてかまいません。

```
$ uv run pc1251
```

2 入れ方と起動

2.1 端末を開く

■ macOS 「アプリケーション」→「ユーティリティ」→「ターミナル」を開きます。Spotlight(`command` + `Space`)で「ターミナル」と打っても開けます。

■ Windows スタートメニューで「PowerShell」と打ち、「Windows PowerShell」を開きます。「ターミナル」(Windows Terminal)が入っていればそれでもかまいません。

2.2 uv を入れる

uv は Python とライブラリをまとめて扱う道具です。一度入れればよく、次からはこの手順は要りません。

■macOS

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

■Windows

```
> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

入れ終わったら、端末をいったん閉じて開き直します (新しく入ったコマンドの場所を端末に読み込ませるため)。次のコマンドでバージョン番号が出れば成功です。

```
$ uv --version
```

Homebrew や winget を使っている場合

macOS で `brew install uv`、Windows で `winget install --id=astral-sh.uv -e` としても同じ uv が入ります。

2.3 エミュレータのフォルダを用意する

GitHub のページから `git clone` で取ってくるか、「Code」→「Download ZIP」でダウンロードした ZIP ファイルを展開して、好きな場所に置いてください。`git clone` で取ってきたときのフォルダの名前は `pc1251-emulator` で、以下でもこの名前を書きます。端末でそのフォルダに移ってからコマンドを打ちます。

■macOS (ホームの src の下に置いた場合)

```
$ cd ~/src/pc1251-emulator
```

■Windows (「ドキュメント」の下に置いた場合)

```
> cd $HOME\Documents\pc1251-emulator
```

フォルダの場所を打つのが面倒なら、`cd` と空白を打ってから、Finder やエクスプローラーのフォルダをターミナルや PowerShell のウィンドウにドラッグ&ドロップすると、場所が入ります。

2.4 ROM を用意する

ROM はシャープの著作物なので、エミュレータには含めていません。自分の実機から読み出したものを使ってください。

必要なのは次の 2 つのファイルです。エミュレータのフォルダの中に `rom` フォルダを作り、この名前で置きます (置き方の例は表のあと)。

ファイル名	中身	大きさ
<code>cpu-1251.rom</code>	CPU(SC61860) の内部 ROM、0000～1FFF 番地	8192 バイト
<code>bas-1251.rom</code>	BASIC ROM、4000～7FFF 番地	16384 バイト

たとえば macOS でホームの `src` の下に置いた場合は、次のようになります。

```
~/src/pc1251-emulator/
  pc1251emu/
  programs/
  rom/
    cpu-1251.rom
    bas-1251.rom
```

`rom` フォルダは、エミュレータのフォルダに移ってから次のコマンドで作れます (macOS でも Windows でも同じです)。Finder やエクスプローラーで ROM のファイルをこのフォルダへ入れます。

```
$ mkdir rom
```

Windows で「ドキュメント」の下に置いた場合は、`C:\Users\名前\Documents\pc1251-emulator\rom\cpu-1251.rom` のようになります。

実機からの読み出し方は、大阪工業大学のサイトにある yagshi 氏のページ「Extracting ROM Data (ROM 吸い出し)」(https://www.oit.ac.jp/rd/labs/kobayashi-lab/~yagshi/old_web/misc/pocketcom/extrom.html) に、PC-1251 を例にした手順があります。BASIC ROM は CSAVE M で区切りながらカセットに保存し、内部 ROM は短いマシン語で外に写してから保存します。録音した音をパソコンでバイナリに戻し、つなげて 1 つのファイルにします。

ROM には版の違いがあるので、エミュレータが確かめるのはファイルの大きさだけです。ファイルが見つからないときや大きさが足りないときは、起動したときにそのことを表示して終わります。

初めて `uv run` を使うとき

最初の `uv run` では、`uv` が Python と、使うライブラリ (`pygame`、`Pillow`、`Typer`) を取ってきて入れるので、しばらく時間がかかります。2 回目からはすぐに始まります。入れたものはこのフォルダの中の `.venv` に置かれ、ほかの場所には何も書き込みません。

ROM を rom 以外の場所に置いておきたいときは、その場所を環境変数 PC1251_ROM に入れておけば、そこから読みます。ほかに、ホームの .pc1251 フォルダも探します。

2.5 起動する

```
$ uv run pc1251
```

PC-1251 の画面がウィンドウに出て、液晶に>(プロンプト) が出れば動いています。最初の 8 秒ほどは、画面の中ほどにキーの説明が出ます (`command` / `Ctrl` + `/` でいつでも出し入れできます)。

画面が大きすぎたり小さすぎたりするときは、拡大率を指定します。画面に収まらないときは自動で縮めます。

```
$ uv run pc1251 --scale 0.8
```

2.6 終わる

ウィンドウを閉じれば終わります。終わるときには、実機のスイッチを OFF にしたのと同じ手順で電源を切ってから、RAM の中身を保存します (4.8 節)。次に起動すると、プログラムも変数もそのまま残っています。

保存したものを使わずにまっさらな状態で始めたいときは `--fresh` をつけます。このときは終わるときにも保存しません。

```
$ uv run pc1251 --fresh
```

2.7 うまく動かないとき

cpu-1251.rom が見つかりませんと出る

ROM が置かれていません。2.4 節を見て、rom フォルダに置いてください。

uv というコマンドがないと言われる

uv を入れたあと、端末を開き直していません。閉じて開き直してください。

Windows で「スクリプトの実行が無効」と出る

uv を入れるコマンドを、上に書いたとおり `-ExecutionPolicy Bypass` をつけて打ってください。

キーを打っても字が入らない

日本語入力になっていると、打ったキーがエミュレータに伝わりません。英数入力に切り替えてください。

Mac で F1～F12 を押しても反応しない

ノート型の Mac では、F1～F12 は明るさや音量のキーになっていることがあります。**fn** を押しながらか、3.4 節の **command** / **Ctrl** + **O** などの組み合わせや右クリックのメニューを使ってください。

音が出ない・音がいらぬ

音を止めたいときは `--mute` をつけて起動します。

3 画面と操作

3.1 各部の名前



液晶

5×7 ドットの文字が 24 桁並びます。点いたドットは濃い青紫で、消えたドットもうっすら見えます。反射型の液晶なので、文字の影が奥に少しずれて落ちて見えます。

表示記号

液晶の上の小さな字です。RUN、PRO、RESERVE(いまのモード)、DEG(角度の単位)、SHIFT(SHIFT を押した)、DEF、BUSY(計算中・実行中)、E(エラー) などが出ます。

モードスイッチ

右上の縦のスイッチです。上から RSV、PRO、RUN、OFF。いちばん上の ON はスイッチの位置ではなく、**BRK** キーが ON(電源を入れる) を兼ねるという表示です。

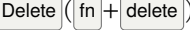
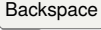
3.2 マウスで操作する

- 画面のキーをクリックすると、そのキーを押したことになります。ボタンを押しているあいだはキーも押しっぱなしです。
- モードスイッチは、ラベル (RSV～OFF) かつまみのあたりをクリックすると、その位置に動きます。つまみを上下にドラッグしても動かさず。

3.3 パソコンのキーボードで操作する

英字・数字・・・矢印キーは、PC-1251 の同じキーとして働きます。押しているあいだは押しっぱなしになります。記号は、パソコンで打った字を PC-1251 での打ち方 (のあとにキー) に直して打ち込みます。たとえばを打つと、 と押したことになります。

PC-1251 の ROM は、1 つのキーを受け取るのに 0.1 秒ほどかかります (キーを離してから次のキーまでに、何も押されていない時間要る)。エミュレータは、パソコンで打ったキーを順番に並べて 1 つずつ届けるので、速く打っても取りこぼしません。毎秒 10 打鍵より速く打った分は、少し遅れて表示されます。

macOS	Windows	PC-1251	説明
英数字・  ・矢印	同じ	同じキー	押しているあいだは押しっぱなし
			
記号	記号	SHIFT+ キー	打った字に合わせて直す (付録付録 B)
			
			
 (fn + )			表示を消す
		 と 	1 字戻って消す
			実行を止める。電源が切れているときは ON

3.4 エミュレータの操作

モードスイッチの切り替えや一覧の表示など、PC-1251 のキーにないエミュレータの操作は、 (Windows と Linux では ) との組み合わせで行います。

macOS	Windows	すること
command + ↑ ↓	Ctrl + ↑ ↓	モードスイッチを1段上・下へ動かす
command + O	Ctrl + O	プログラムの一覧 (5 節)
command + R	Ctrl + R	裏の RESET ボタンを押す
command + T	Ctrl + T	速さを1倍と4倍で切り替える
command + E	Ctrl + E	BASIC のプログラムをファイルに書き出す (5.9 節)
command + S	Ctrl + S	画面を画像で保存する
command + /	Ctrl + /	キーの説明を出す・消す
command + V	Ctrl + V	クリップボードの文字を打ち込む

同じことは、ウィンドウの上で右クリックして出るメニューからでもできます (Mac では **control** を押しながらクリックしても出ます)。メニューの項目はクリックするか、↑ ↓ で選んで **Enter** で実行します。いちばん上の「プログラムを読み込んで実行」を選ぶと、メニューがプログラムの一覧 (5 節) に変わり、選んだものをそのまま読み込んで実行します。



ファンクションキー

F1～**F4** (スイッチの RUN、PRO、RSV、OFF)、**F5** (RESET)、**F6** (一覧)、**F9** (速さ)、**F10** (説明)、**F12** (画面の保存) も使えます。ただ、Mac では **fn** を押しながらでないと明るさや音量のキーになり、Windows でもほかのアプリに取られることがあるので、ふだんは上の組み合わせか右クリックのメニューを使ってください。

command / **Ctrl** + **S** で保存した画像は、ホームの **.pc1251** フォルダに **shot001.png**、**shot002.png**……の名前で置かれます (Windows では **C:\Users\名前\.pc1251**)。

3.5 モードスイッチ

モード することと表示

RUN	計算とプログラムの実行。表示記号は RUN。
PRO	プログラムを打ち込む・直す・LIST で見る。表示記号は PRO。
RSV	よく使う文字列をキーに登録する (4.6 節)。表示記号は RESERVE。
OFF	電源を切る。プログラムと変数は残る。

3.6 SHIFT と DEF

SHIFT は、押して離してから次のキーを押します (同時に押す必要はありません)。押すと表示記号 **SHIFT** が点き、次のキーがキーの上に刷ってある字 (朱色の字) になります。たとえば **SHIFT W** で"、**SHIFT 0** で π です。

DEF は、プログラムの中の "A" のような印の行から実行を始めるキーです (4.5 節)。

4 使ってみる

4.1 電卓として使う

RUN モードで式 (たとえば 12×3) を打って **ENTER** を押すと、答えが出ます。

12*3



CL (パソコンでは **Delete**) で表示が消えます。

4.2 BASIC のプログラムを打ち込んで動かす

1. スイッチの PRO をクリックするか、**command** / **Ctrl** + **↑** で PRO モードにします。
2. 前のプログラムを消したければ **NEW** と打って **ENTER** を押します。
3. 行番号をつけて、1 行ずつ打ち込みます。行の終わりで **ENTER** を押します。

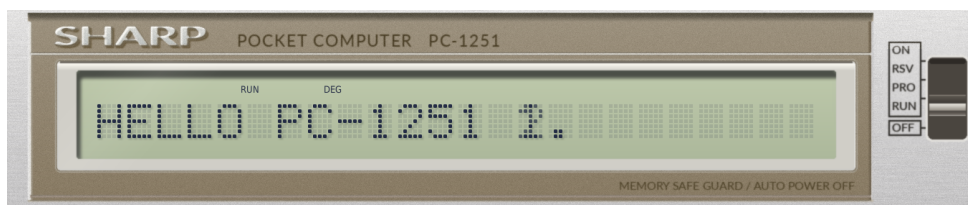
```
10 FOR I=1 TO 3
20 PRINT "HELLO PC-1251 ";I
30 NEXT I
```

40 END

4. LIST と打つと、先頭の行が出ます。↓↑で前後の行に動きます。



5. command / Ctrl + ↓ で RUN モードに戻り、RUN と打って ENTER を押します。



PC-1251 の PRINT は、表示したところで止まって待ちます。ENTER を押すと先へ進みます (止めに進める方法は次の節)。実行を途中で止めるには BRK (パソコンでは Esc) を押します。

長いプログラムを手で打つのは大変なので、ふつうはテキストファイルに書いておき、5 節の方法で読み込むのが楽です。

ENTER のあとは少し待つ

実機と同じく、ENTER を押したあとは ROM が行を中間コードに直すのに時間がかかります。その間に次のキーを押すと取りこぼすことがあります。ファイルから読み込むときは、エミュレータがこの待ち時間をもって打ち込みます。

IF のあとの代入には LET がいる

IF の条件のあとに代入を書くときは、LET を省けません。THEN をつけても同じです。

書き方	結果
IF S>H LET H=S	代入される
IF S>H THEN LET H=S	代入される
IF S>H THEN H=S	ERROR 4
IF S>H H=S	エラーは出ないが、代入されない

最後の書き方は止まらずに進んでしまうので、間違いに気づきにくいところです。代入でなければ、IF A=1 GOTO 50 や IF D=1 POKE &C5DF,1 のように、条件のすぐあとに命令を書けます。

4.3 打ち込みを楽にする、行を直す・消す

■命令は略して打てる 命令の頭の何文字かのあとに.(ピリオド)を打つと、その文字で始まる命令になります。たとえば P."AB"と打てば PRINT "AB"と同じです。PRO モードでも RUN モードで

も使え、LIST すると元の綴りで表示されます。

略記	命令	略記	命令	略記	命令
P.	PRINT	G.	GOTO	GOS.	GOSUB
I.	INPUT	F.	FOR	N.	NEXT
RE.	RETURN	REA.	READ	RES.	RESTORE
R.	RUN	L.	LIST	T.	THEN
B.	BEEP	W.	WAIT	CA.	CALL
D.	DIM	E.	END	S.	STOP

同じ文字で始まる命令がいくつかあるときは、ROM の命令の表で先に載っているものになります。C. は CALL ではなく CONT に、RE. は READ ではなく RETURN になるので、ほしい命令にならないときは文字を 1 つ増やします。.bas のファイルに略記で書いても同じように読み込め、5.9 節の方法で書き出すと元の綴りに戻ります。

■行を消す PRO モードで行番号だけを打って **ENTER** を押すと、その行が消えます。たとえば 20 **ENTER** で 20 行が消えます。プログラム全体を消すときは NEW です。同じ行番号で打ち直すと、前の行と入れ替わります。

■行を直す PRO モードで LIST 30 のように行を出し、**▶** を押すと、行番号のすぐあとの文字にカーソルが出ます。

- **◀▶** でカーソルを動かし、文字を打つとカーソルの位置の文字を上書きします。
- **SHIFT ▶** (INS) でカーソルの位置に空きを作り、そこへ文字を差し込みます。
- **SHIFT ◀** (DEL) でカーソルの位置の文字を消します。
- 直し終わったら **ENTER** で、その行が入れ替わります。
- **↓↑** で次の行・前の行を出せます。

■パソコンのキーボードで打つとき **Backspace** (macOS では **delete**) を押すと、エミュレータが **◀** と **SHIFT ◀** (DEL) を続けて押し、カーソルの 1 つ前の文字を消します。打ち間違えたらそのまま **Backspace** で戻れます。矢印キーは **◀▶↑↓**、**Delete** (macOS では **fn + delete**) は **CL** (表示を消す) です。

4.4 PRINT で止めずに見せる (WAIT)

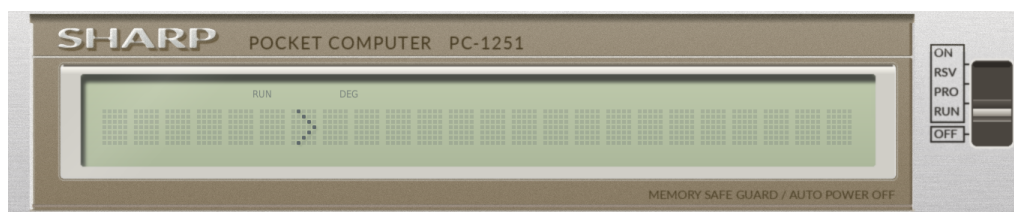
PRINT のたびに **ENTER** を押すのでは、表示を動かすことができません。WAIT を使うと、PRINT は決まった時間だけ表示してから、止まらずに次へ進みます。

書き方	PRINT の動き
WAIT 64	表示してからおよそ 1 秒待ち、次へ進む。数は 10 進法で、大きいほど長く見せる
WAIT 0	待たずにすぐ次へ進む。表示は目に見えないほど短い
WAIT	数を書かないと、 ENTER を待つもとの動きに戻る

一度 WAIT を実行すると、そのあとの PRINT はすべてその設定で動きます。次の例は、> を 1 字

ずつ右へ動かし、最後に GOAL と出して止まります (見本の anime.bas)。

```
10 DIM S$(0)*24
20 WAIT 6
30 FOR I=1 TO 20
40 PRINT S$(0); ">"
50 S$(0)=S$(0)+" "
60 NEXT I
70 WAIT:PRINT "GOAL"
80 END
```



`S$(0)` は、前に置く空白をためておく文字列です。ふつうの文字列変数 (`A$` など) には 7 字までしか入らないので、10 行の `DIM` で 24 字まで入る配列にしています。20 行の `WAIT 6` で、1 こまが 4 分の 1 秒ほどになります (待ち時間のほかに、計算と表示の時間がかかる)。

PC-1251 の BASIC には、液晶の点を 1 つずつ描く命令がありません。動きのある表示は、このように文字を `PRINT` し直して作ります。点で絵を描きたいときは、液晶の RAM に直接書くマシン語を使います (5.6 節の正弦波やブロック崩しの見本)。液晶に 1 列ずつ点を打てる PC-インタープリタ (5.7 節) の `DEG` も使えます。PC-インタープリタの `PRINT` は止まらないので、`WAIT` は時間待ちの命令として使います。

4.5 DEF キーで実行する

行の頭に "A" のような印をつけておくと、RUN モードで **DEF A** と押すだけで、その行から実行できます。

```
10 "A" PRINT 5
20 END
```

DEF A で 5 が出ます。印に使えるのは、RSV モード (4.6 節) と同じページの帯の上の 18 個のキー (**A** ~ **L**、**=**、**Z** ~ **M**、**SPC**) です。

4.6 RSV モードで文字列を登録する

よく打つ BASIC の命令や式を、キーに登録しておけます。

1. スイッチの RSV をクリックして RSV モードにします。表示記号が RESERVE になります。

2. **SHIFT A** と押すと、液晶に **A:** と出ます。
3. 登録したい文字列 (たとえば **PRINT** や **12*3**) を打って、**ENTER** を押します。

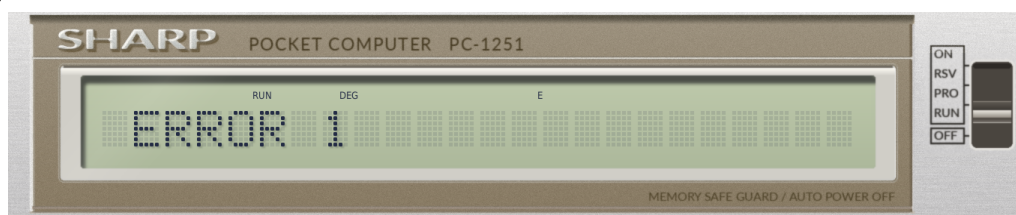


4. **RUN** モードか **PRO** モードで **SHIFT A** と押すと、登録した文字列が打ち込まれます。
登録に使えるのは、キーボードの下 2 段にある、ページュの帯の上のキーです。**A** ~ **L** と **=**、**Z** ~ **M** と **SPC** の 18 個で、上の段のキー (**Q** など) は使えません。帯の色がそのしるしになっています。

4.7 エラーが出たら

打ち込んだ式やプログラムにまちがいがあると、**ERROR** と番号が出て、表示記号 **E** が点きます。

CL で消えます。



RUN モードで **PRINT 3** と打つと **ERROR 1** になる

RUN モードで **PRINT** を式の前につけて直接打つと、**ERROR 1** になります。これはエミュレータのまちがいではなく、ROM がそう動きます (ROM の 56BA 番地で状態を見て止めています)。電卓として使うときは **12*3** のように式だけを打ちます。プログラムの中の **PRINT** は、ふつうに動きます。

4.8 電源とメモリ

電源を切る

スイッチの **OFF** をクリックします。スイッチが **OFF** になったことを ROM が読んで、メモリが生きている印を書いてから自分で電源を切ります。

電源を入れる

スイッチを **RUN** か **PRO** に戻します。

オートパワーオフ

何もしないで 11 分ほどたつと、電源が切れます (実機と同じ)。**BRK** (**ON**) を押せば入ります。プログラムは残っています。

プログラムが残るしくみ

PC-1251 は、電源を入れたときに CPU の中のメモリに書いた印を確かめ、印がなければメモリがこわれたとみなして中身を消します。エミュレータは終わるときにも ROM に電源を切らせ、

RAM(4KB) と CPU 中のメモリ (96 バイト) の両方を、ホームの `.pc1251/ram.bin` に保存します。

RESET ボタン

`command` / `Ctrl` + `R` で本体の裏の RESET ボタンを押せます。プログラムが暴走して `BRK` でも止まらないときに使います。

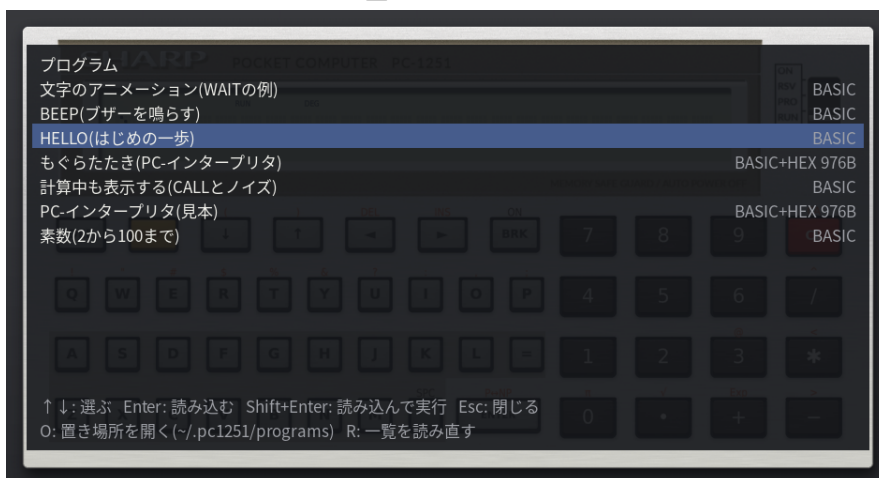
5 プログラムのファイルを使う

5.1 置き場所に置いて、一覧から読み込む

BASIC のプログラムやマシン語のダンプを、決まったフォルダ (置き場所) にファイルとして置いておくと、`command` / `Ctrl` + `O` で開く一覧に出てきます。

macOS	/Users/名前/.pc1251/programs
Windows	C:\Users\名前\.pc1251\programs

このフォルダは、最初に一覧を開いたときにできます。一覧で `O` を押すと、Finder やエクスプローラーでこのフォルダが開くので、そこへファイルを入れてください。名前が「`.`」で始まるフォルダは、ふだんは隠れていて見えにくいので、`O` で開くのが確実です。



キー	すること
<code>↑</code> <code>↓</code>	選ぶ (マウスでクリックしても選べる)
<code>Enter</code>	読み込む (選んであるものをもう一度クリックしても同じ)
<code>Shift</code> + <code>Enter</code>	読み込んで実行する
<code>O</code>	置き場所のフォルダを開く
<code>R</code>	一覧を読み直す (ファイルを足したあと)
<code>Esc</code>	一覧を閉じる

置き場所のほかに、エミュレータのフォルダの `programs` にある見本 (HELLO、BEEP、素数、

文字のアニメーション、計算中の表示、正弦波、ブロック崩し、PC-インタープリタ、もぐらたたき) も一覧に並びます。同じ名前のファイルが両方にあるときは、自分の置き場所のほうが使われます。

■ドラッグ&ドロップ Finder やエクスプローラーからファイルを PC-1251 のウィンドウにドロップしても、同じように読み込めます。

5.2 ファイルの形式

ファイル	読み込み方
名前.bas	BASIC のテキスト。PRO モードにして NEW してから 1 行ずつ打ち込み、もとのモードに戻す。
名前.hex	16 進ダンプ。番地 データの行を読んで、RAM にそのまま書き込む。
名前.bas と名前.hex	同じ名前の 2 つは 1 本のプログラムとして扱い、ダンプを書いたから BASIC を打ち込む。マシン語と、それを呼ぶ BASIC を組にしておく。

■BASIC のファイル ふつうのテキストファイルです。文字コードは UTF-8(ASCII の範囲なら何でも同じ) で、改行は macOS・Windows どちらの形でもかまいません。小文字で書いても大文字に直して打ち込みます。PC-1251 のキーにない字は捨てます。空の行と、#で始まる行は打ち込みません。

```
# title: 素数 (2 から 100 まで)
# PRINT で止まるので、ENTER で次を出す
10 PRINT 2
20 FOR N=3 TO 100 STEP 2
30 D=3
40 IF D*D>N GOTO 70
50 IF N=INT (N/D)*D GOTO 80
60 D=D+2:GOTO 40
70 PRINT N
80 NEXT N
90 END
```

■16 進ダンプのファイル 1 行に番地とデータを 16 進法で書きます。番地は 4 桁です。データは空白で区切っても続けて書いてもかまいません。雑誌のダンプリストのように行の終わりに: とチェックサムがついていれば、そこから先は読み飛ばします。#か; で始まる行はコメントです。行の途中でも; から後ろはコメントなので、ニーモニックを書き添えておけます。

```
# title: マシン語の見本
C300 02 41 37
C303 0102030405060708 ; 表の中身
```

■指示の行 #で始まる行には、次の2つの指示を書けます。`.bas` と `.hex` のどちらに書いてもかまいません。

書き方	意味
# title: 名前	一覧に出す名前。書かなければファイル名。
# run: 文字列	<code>Shift+Enter</code> や <code>--run</code> で実行するときに、RUN モードで打つ文字列。書かなければ RUN。
# hex: ファイル名	先に書き込む 16 進ダンプ。同じフォルダか置き場所から探す。1 つのマシン語を、いくつかの BASIC のプログラムから使える。

読み込みの速さ

BASIC は、実機でキーを押すのと同じ打ち込み方で入れます (行を中間コードに直すのは ROM の仕事です)。読み込むあいだは実機の時計に合わせず全速で動かすので、数十行のプログラムでも 1~2 秒で終わります。画面の下に進み具合が出ます。

NEW で前のプログラムは消える

`.bas` を読み込むと、その前に NEW するので、入っていたプログラムは消えます。残しておきたいプログラムは、先に 5.9 節の方法でファイルに書き出しておいてください。

5.3 端末から読み込む

起動するときにプログラムの名前かファイルの場所を書くと、起動してすぐに読み込みます。名前だけなら置き場所から探します。`--run` をつけると、読み込んだあとと実行します。

```
$ uv run pc1251 primes --run
$ uv run pc1251 ~/basic/game.bas
```

Windows なら、ファイルの場所は `C:\Users\名前\basic\game.bas` のように書きます。

5.4 マシン語を動かす

マシン語は 16 進ダンプのファイルにして RAM に書き込み、BASIC の `CALL` で呼びます。PC-1251 の RAM は B800~C7FF 番地の 4KB で、BASIC のプログラムは下の番地から、変数は上の番地から使います。マシン語は、BASIC のプログラムや変数とぶつからない番地に置いてください。

5.5 CALL で表示をつけたまま計算する

BASIC はプログラムを実行しているあいだ、液晶の表示を切っています (C ポートのビット 0 を 0 にする)。マシン語でこのビットを 1 にしてから計算を続けると、実行中も表示が消えません。見本の `noise.bas` は、そのためのマシン語 5 バイトを POKE で書いて `CALL` で呼び、計算しながら液晶に棒を描きます。

```

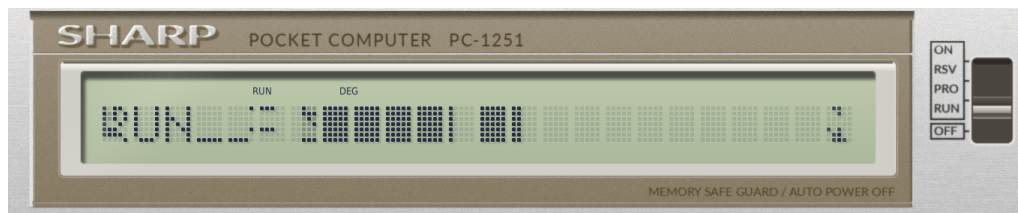
10 POKE &C200,2,1,255,177,55
20 CALL &C200
30 FOR I=0 TO 47
40 B$=STR$ (I*I)
50 POKE &F87B-I,127
60 NEXT I
70 WAIT 128:PRINT "DONE"
80 END

```

10 行で書くマシン語は次の 3 命令です。1FB1 番地は CPU の内部 ROM にあるルーチンで、A の値を C ポートに出します。

C200	02 01	LIA 01	A=&01(表示をオン)
C202	FF B1	CAL 1FB1	C ポートに A を出す
C204	37	RTN	BASIC に戻る

50 行は、液晶 RAM に直接 1 列ずつ点を書きます。液晶の右半分は F87B 番地が左端 (60 列目) で、番地が小さくなるほど右へ進むので、&F87B-I で棒が右へ伸びます。



表示をつけたままにすると、左の数桁にノイズのような模様が出ます。これは ROM が液晶 RAM の一部を計算の作業場所に使っているためです。40 行の STR\$ は数を文字にする途中で F800~F822 番地 (左から 7 桁ほど) を書き換え、ほかの計算でも右端の F840~F847 番地などを使います。ふだんは実行中に表示を切っているので、この書き換えは目に見えません。

CALL するマシン語の置き場所

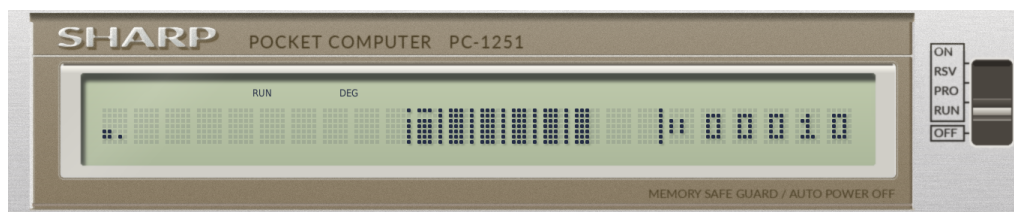
C200 番地は正弦波の見本や PC-インタープリタ (5.7 節) と同じ場所です。両方を続けて使うときは、あとから読み込んだ方が残ります。

5.6 マシン語で液晶に描く

■正弦波 見本の wave.hex は、液晶に正弦波を流すマシン語です (C200~C241 番地のプログラムと、C300 番地からの波の形の表)。1 行ごとに ; のあとへニーモニックを書いてあります。同じ名前の wave.bas が CALL &C200 で呼ぶので、一覧から wave を選ぶと両方が読み込まれます。表から 60 列ぶんを 1 コマごとにずらして液晶 RAM に写し、Esc (BRK) を押すと BASIC に戻ります。

■ブロック崩し 見本の `break.hex` と `break.bas` は、横向きのブロック崩しです。左端のパドルを `↑` `↓` で動かし、`Space` か `Enter` で玉を打ち出して、右のブロックの壁を削ります。壁と画面の右端のあいだは空いているので、そこへ玉を回り込ませると壁を裏から削れます。壁はゆっくり左へ寄ってきて、先端が左から 12 列目まで来ると押し切れられ、玉を 1 個失います。玉は 3 個で、全部なくなると BASIC に戻って得点を出します。右端の 3 つの点が残りの玉、その右の 5 桁が得点です。

始めに 1:EASY 2:NORMAL ? と聞いてくるので、`1` か `2` を押して `ENTER` を押します。EASY はパドルが縦 3 段で、玉は 1 コマに 1.5 列、壁は 60 コマで 1 列と遅めです。NORMAL はパドルが縦 2 段で、玉は 1 コマに 2 列、壁は 40 コマで 1 列です。`ENTER` だけを押すと前と同じ難しさで始めます。最高点は難しさごとに覚えています。



パドルに当たった場所で玉の角度が変わり、パドルを動かしながら当たるとさらに角度がつきます。縦が 7 ドットしかないので、同じ角度のまま打ち返していると、同じ段ばかりを往復して最後の数個が残りがちです。角度を変えて狙ってください。面をクリアすると 500 点が入り、次の面から玉が少し速くなります。

マシン語は C000~C4AA 番地の約 1 KB で、液晶に描く前の絵を C501 番地からの 120 バイトに組み立ててから、液晶の RAM へまとめて写します。ROM のルーチンは呼ばず、キーは IA・IB のポートを直接読み、音は C ポートのビット 4~6 を切り替えて出します。内部 RAM の 10h~2Fh を変数に使うので、入るときに BASIC の値を C5E0 番地からの 32 バイトへ退避し、出るときに入れ替えて戻します。このため終わったあとも変数が退避先に残り、得点の 5 桁は C5F6~C5FA 番地から PEEK で読めます。`break.bas` はこれを使って得点と最高点を出します。難しさは、CALL の前に C5DF 番地へ POKE しておきます (1 なら EASY)。

5.7 PC-インタープリタともぐらたたき

見本には、PiO 1986 年 8 月号の「PC-インタープリタ」(Fan_PC-1251 氏作)が入っています。BASIC のエディタで書いた行を独自の文法で読んで実行するマシン語のプログラムで、数はすべて 16 進法 2 桁の 1 バイト、命令は PRINT、IF~NOT、GOTO、FOR~NEXT、BEEP、WAIT、液晶に 1 列ずつ点を打つ DEG などです。

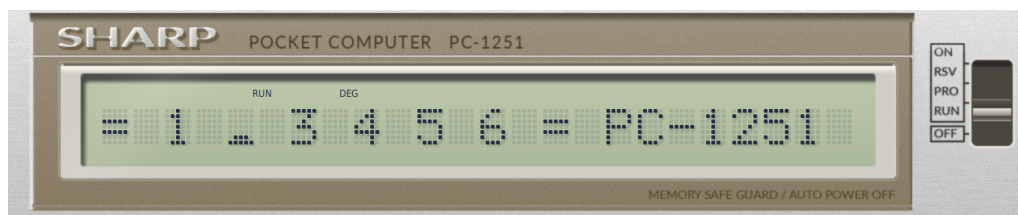
`pcint.hex` と `pcint.bas`

記事のリスト 1 (C200~C5CF の 976 バイト) と、それを試す短いプログラムの組です。`Shift` + `Enter` で読み込むと、CALL &C300 で動き出します。

`mogura.bas`

記事のリスト 2 のもぐらたたきです。# hex: `pcint.hex` の行で PC-インタープリタを先に書

き込み、RUN 900 で始めます。



もぐらたたきを始めるには、ウィンドウを右クリックして「プログラムを読み込んで実行」→「もぐらたたき (PC-インタープリタ)」を選びます。command / Ctrl + O の一覧で選んで Shift + Enter を押しても同じです。PC-インタープリタのダンプとリスト 2 が読み込まれ、RUN モードで RUN 900 が打たれてゲームが始まります。一度読み込んだあとは、RUN モードで RUN 900 と打つか、DEF = を押せば何度でも遊べます (900 行の頭に "=" の印がある)。

900 行は CALL &C300 で PC-インタープリタを呼び、ゲームが終わって BASIC に戻ってから PRINT で得点を出します。CALL &C300 とだけ打っても遊べますが、そのときは得点が出ません。

遊び方は次のとおりです。液晶に 1~6 の穴が並び、どこかの穴からもぐらが 3 段階で顔を出します。出た穴の数字キーを押しっぱなしにしておくと、当たりの音が鳴って得点が 1 つ増え、次のもぐらが出ます。当てるたびに少しずつ速くなります。3 段階まで出きても押せないと GAME OVER になり、ENTER を押すと得点が出ます。

記事のプログラムの扱い

pcint.hex と mogura.bas は記事の作者 (Fan_PC-1251 氏) の著作物で、作者の了解を得て収めています。このプログラムを 1 命令ずつ読み解いた『PC-インタープリタを読む』(おりぐち) も doc/pcinterp.pdf にあります。

5.8 そのほかの読み込み方

command + V (Windows は Ctrl + V)

クリップボードの文字を、そのままキーとして打ち込みます。NEW もモードの切り替えもしないので、打ち込む前に PRO モードにしておきます。

--type ファイル

起動したあと、ファイルの中身をそのままキーとして打ち込みます。

--load ファイル

起動したあと、16 進ダンプを RAM に書き込みます。

5.9 プログラムをファイルに書き出す

エミュレータで打ち込んだり直したりした BASIC のプログラムは、RAM の中にしかありません。

command / Ctrl + E を押すか、右クリックのメニューで「プログラムをファイルに書き出す」を選べると、いまのプログラムをプログラムの置き場所 (~/.pc1251/programs) に .bas のファイルとして書き出します。ファイル名は「名前-日時.bas」で、名前は最後に読み込んだプログラムのもの (な

ければ program) です。書き出したファイルはすぐに一覧に出るので、そのまま読み込み直せます。

RAM のプログラムは中間コードになっているので、ROM の中にある命令の表を引いて文字に戻します。空白は ROM が打ち込みのときに捨てているので、命令の前後に読みやすいように補います。打ち込み直せば、元と同じ中間コードになります。

読み込んだプログラムに組になる .hex があったときは、書き出したファイルの頭に # hex: の行を入れて、元のダンプを指しておきます。POKE で書いたマシン語など RAM の中身そのものは書き出しません。

エミュレータを起動しないで、保存してある RAM から書き出すこともできます。

```
$ uv run pc1251 --export ~/Desktop
```

付録 A 起動のオプション

オプション	意味
プログラム (名前)	読み込むプログラム (ファイルの場所か、置き場所にある名前)
--run	読み込んだあと実行する
--scale 数	拡大率 (既定は 1.0。画面に収まらなければ縮める)
--fresh	保存した RAM を読まず、終わるときにも保存しない
--mute	音を出さない
--persist 数	液晶の残像の強さ (既定は 1.0、0 で残像なし)
--type ファイル	起動後にファイルの中身をキーとして打ち込む
--load ファイル	起動後に 16 進ダンプを RAM へ書き込む
--export フォルダ	保存した RAM の BASIC のプログラムを .bas で書き出して終わる
--help	オプションの一覧を出す

環境変数	意味
PC1251_ROM	ROM を置いたフォルダ
PC1251_PROGRAMS	置き場所として先に探すフォルダ。いくつも書くときは、macOS では:、Windows では; で区切る

付録 B SHIFT で入る記号

パソコンで次の記号を打つと、右のキー操作に直して打ち込みます。

記号	キー	記号	キー	記号	キー
!	SHIFT+Q	&	SHIFT+Y	(	SHIFT+ ↓
"	SHIFT+W	?	SHIFT+U	)	SHIFT+ ↑
#	SHIFT+E	:	SHIFT+I	@	SHIFT+3
\$	SHIFT+R	,	SHIFT+O	<	SHIFT+*
%	SHIFT+T	;	SHIFT+P	>	SHIFT+-
^	SHIFT+/	√	SHIFT+.	π	SHIFT+0

SHIFT + は指数の E(Exp)、**SHIFT ◀** は 1 字消す (DEL)、**SHIFT ▶** は 1 字あける (INS)、**SHIFT ENTER** は $P \leftrightarrow NP$ (プリンタを使うかどうかの切り替え)、**SHIFT CL** は CA (すべて消す) です。

付録 C しくみのあらまし

番地	中身
0000-1FFF	CPU の内部 ROM(8KB)
4000-7FFF	BASIC ROM(16KB)
B800-C7FF	RAM(4KB)。B000-B7FF は B800-BFFF と同じものが見える
F800-F87F	液晶の RAM

- CPU は SC61860 で、命令サイクルは 192kHz(水晶 576kHz の 1/3) です。命令の意味は MAME の実装を下敷きにしています。
- モードスイッチは、ポート IB のビット 3 を出して INB 命令で読みます。ビット 0 が RSV、ビット 1 が PRO、ビット 2 が OFF で、どれも立っていなければ RUN です。
- C ポートのビット 2 を立てると、CPU はキーが押されるか 512ms のタイマが変わるまで止まって待ちます。ROM はキーを待つあいだこの方法で止まり、起きた回数を数えてオートパワーオフの時間を計ります。
- ブザーは C ポートのビット 4~6 で鳴らします。BASIC の BEEP は内蔵の 4kHz の発振を使い、マシン語ではビットを直接切り替えて好きな高さの音を出せます。

付録 D 実機と違うところ

- 実機と並べて動作を照らし合わせてはいません。命令の解釈、キーの配置、ポートの意味は、MAME、PockEmul、POPCOM 1984 年 9~11 月号のマシン語講座と、ROM の読み解きによります。
- カセットテープの入出力は、動作を確かめていません。CSAVE はそれらしい音が出ますが、録音しても実機で読めるとは限りません。CLOAD は **BRK** (**Esc**) で止めてください。
- プリンタ (CE-125) は動きません。
- ROM の版の違いは確かめていません。
- 内部 ROM が BASIC の PEEK でも読めます。実機では DATA(35h) 命令でしか読めません。

付録 E ファイルの一覧

場所	中身
pc1251emu/	エミュレータ本体 (CPU、メモリとポート、画面、音、プログラムの読み込み)
programs/	見本のプログラム
rom/	自分で用意した ROM(2.4 節)
tests/	動作の試験 (uv run python tests/test_basic.py など)
doc/	この手引きと『PC-インタープリタを読む』(pcinterp.pdf)
~/.pc1251/	保存した RAM、画面の画像、プログラムの置き場所